

1. Explain the rationale for the project and whether it had the expected impact.

Project Name: Mission Control - One Click Deployment for Blockchain Infra

About Blockops

Blockops is a web3 infrastructure platform that makes it easy for anyone to deploy and manage Blockchain Nodes anywhere (on cloud or on baremetal servers). Think DigitalOcean but for Blockchain Infra

Objective

As Blockops expanded its support for more blockchain networks and protocols, onboarding a new protocol took approximately 12 weeks of cross-functional effort across Infrastructure, backend, and frontend teams. With over 120 active blockchains to integrate as potential customers, this integration timeline became a problem for the business and sales team

Internal OKR & KPI tracking exposed the gaps:

- Delays in the Enterprise sales cycle
- Each integration required manual coordination with the infra team to provision base infra, observability, elastic storage, and security hardening configurations specific to each blockchain network.
- Engineering capacity was increasingly spent on repetitive integration work rather than core platform and continuous improvements on existing products

As the Lead of the Platform team, I was responsible for delivering platform products for the business. It was my responsibility to identify operational gaps and resolve them. In this case, I identified the need to reduce network integration time while maintaining reliability and multi-environment compatibility (cloud and on-premise).

Success Metrics (from my Performance Evaluation Document)

- Reduce network integration time for Mission Control from 12weeks to 1week
- Standardize deployment patterns across cloud / bare metal platforms
- Maintain a 99.99% SLA for all Mission Control components

My strategy was to design a bootstrapable infrastructure that automated all repetitive components of our setup with complete correctness. I began by creating what I called a "reference architecture module." This served as the base module where shared infrastructure

components such as networking, storage configuration, and release automation resided. Each unique blockchain module was then built on top of this foundation.

Key Decision Made

- Choosing the right infra-as-code tooling between Pulumi and Terraform
- Designing for multi-cloud versus on-prem compatibility from the outset
- Deciding how much abstraction to introduce without hiding operational complexity.
- Handling deployment concurrency and scaling constraints

Final Outcome:

- Integration time reduced from 12 weeks to 2 weeks (83% reduction).
- Engineering effort per integration reduced by approximately 60%.
- Enabled significantly faster onboarding of new networks.
- Improved consistency across environments

However, post-launch we discovered deployment concurrency limitations in the selected tooling, which required architectural adjustments (internal queuing with RabbitMQ). Despite this, the project materially improved delivery velocity and platform scalability.

2. Thought Process for Decisions

One significant decision was selecting Pulumi over Terraform as the infrastructure-as-code framework. I considered different options, including using Terraform directly and building a Go wrapper around the Terraform open-source code versus using Pulumi, which already provides multi-language SDK support. The primary barrier to adopting Pulumi at the time was that I was less experienced with it compared to Terraform, which introduced a learning curve.

In my architectural design record, I already listed out the pros vs cons of each approach and evaluated them using the following criteria:

- Team accessibility across backend, frontend, and infrastructure engineers
- Ability to create reusable abstractions
- Maintainability
- Long-term scalability
- Learning curve

Terraform was mature and widely adopted, but required writing HCL, which was unfamiliar to some engineers. Pulumi allowed infrastructure to be written in general-purpose programming languages (e.g., TypeScript, Go), which aligned with the team's existing skill set.

I prioritized cross-functional contribution. Since backend engineers could contribute directly using familiar languages, Pulumi reduced siloing and accelerated iteration during early development.

To de-risk the decision:

- We ran a proof-of-concept deployment across multiple environments.
- We stress-tested parallel deployments.

In retrospect, some part of the POC we didn't cover that later came to bite us:

- Evaluation of state management under load
- Reviewing licensing implications

Outcome:

Pulumi enabled faster development velocity and broader team participation during the build phase. However, once adoption increased, we encountered concurrency limitations that restricted the number of parallel deployments without upgrading to an enterprise tier.

To mitigate this without increasing cost, I designed an internal deployment queue using RabbitMQ to serialize user-triggered deployments and avoid state conflicts. This preserved system stability while we evaluated long-term options.

In hindsight, while Pulumi accelerated development, I would have performed deeper concurrency modeling earlier to anticipate scaling constraints

3. How did you approach delivery in this project?

I approached delivery in three phases: Architectural Designs & Scoping, Reference Architecture Build out with reusable modules, and Scaling, Hardening & Testing

Phase 1: Architectural Designs & Scoping

I created an Architectural Decision Record (ADR) documenting:

- The problem (The Why)
- Available options (The What)
- Tradeoffs

- Selected approach (The How)
- Long-term implications
- Delivery Timelines (The When)

This ensured alignment across engineering functions and reduced opinion-driven debates.

Phase 2: Building Reference Architecture modules

I identified reusable modules and scoped them accordingly:

- Base infrastructure (networking, compute, storage)
- Node deployment modules
- Monitoring stack
- Secret management
- Environment abstraction (cloud vs on-prem)

Rather than delivering a fully abstracted system at once, we prioritized delivering a minimal viable integration for one network first to validate assumptions.

Phase 3: Scaling & Hardening

After validating the model with one network, we generalized modules for additional protocols and improved deployment automation.

We intentionally prioritized backend and infrastructure automation first, accepting that frontend automation would follow shortly after. This resulted in integration time reducing to 2 weeks rather than the initial 1-week goal.

Relative to expectations:

- Velocity improvements exceeded expectations.
- Concurrency scaling issues emerged later than anticipated.
- Cross-team collaboration improved significantly due to shared ownership of infrastructure code.

Overall, the phased delivery approach reduced risk and allowed iterative refinement.

4. What were your biggest individual learnings? How would you apply them if given the chance to redo that project?

The most significant learning was the importance of modeling scale constraints early, even when solving an immediate velocity problem.

While the project successfully reduced integration time by 83%, I underestimated how quickly deployment concurrency would grow. This resulted in reactive architectural adjustments (internal queuing) rather than proactive design.

I also learned that infrastructure tooling decisions should be evaluated not only for development ergonomics, but also for operational economics (licensing tiers, scaling limits, long-term cost implications).

If I were to redo the project, I would:

- Conduct deeper load and concurrency modeling before finalizing tooling.
- Evaluate long-term cost implications under projected growth scenarios.
- Introduce deployment queueing as a native architectural component rather than as a corrective measure.

The project strengthened my approach to balancing speed, abstraction, and operational realism.

5. LLM Disclosure

Yes. I used an LLM to assist with editing and clarity. I initially drafted all responses independently based on my direct experience. I then used the LLM to:

- Correct grammar and spelling
- Improve clarity and structure
- Ensure conciseness within word limits

The project details, decisions, and analysis reflect my own work and judgment.

Prompt: "You're a professional proofreader. Refine this draft for grammar, clarity, and conciseness. Do not change the original content or tone or add new content."